TECHNICAL DOCUMENT

Local Development Guide

docs/LOCAL_DEV_GUIDE.md

Setup · Docker · Build · Run

PLATFORM	STACK	GENERATED
EduTech v1.0	Java 17 · Spring Boot 3.3.5	March 07, 2026

Local Development Guide

This guide walks through setting up the EduTech AI Platform on a local machine from a fresh clone to a fully running stack. Follow steps in order — each step depends on the previous one completing successfully.

Table of Contents

1. Prerequisites
2. Environment Setup
3. RSA Key Generation (JWT)
4. Starting Infrastructure
5. Database Initialization
6. Building the Project
7. Running Services
8. Service Ports Reference
9. Verifying the Stack
10. Dev GUI Tools
11. Common Issues and Fixes

Prerequisites

TOOL	MINIMUM VERSION	NOTES
Java JDK	17	Must be a JDK, not a JRE. Eclipse Temurin is recommended. Install via SDKMAN: `sdk install java 17.0.x-tem`
Maven	3.9	Verify: `mvn -version`. Ensure `JAVA_HOME` points to your JDK 17 installation.
Docker Desktop	4.x	Compose V2 plugin is required (`docker compose` — no hyphen). Verify: `docker compose version`
Git	Any recent	

Verify your setup before proceeding:

```
java -version          # expect: openjdk 17.x.x
mvn -version           # expect: Apache Maven 3.9.x, Java version: 17.x
docker compose version # expect: Docker Compose version v2.x.x
```

> **macOS note:** If you installed Java via Homebrew or Adoptium, add `export JAVA_HOME=$(/usr/libexec/java_home -v 17)` to your shell profile so Maven consistently picks up JDK 17.

Environment Setup

1. Clone the repository

```
git clone https://github.com/your-org/school-APK.git
cd school-APK
```

2. Create your local `.env` file

```
cp .env.example .env
```

`.env` is listed in `.gitignore` and is never committed. `.env.example` is the committed canonical reference — every variable is documented there with inline comments.

3. Fill in required values

Open `.env` in your editor. The table below identifies which variables are **required for local development** versus which can be deferred or left blank.

PostgreSQL

```
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_ROOT_USER=edutech_admin         # required
POSTGRES_ROOT_PASSWORD=localpass         # required — any non-empty string

# Per-service credentials (repeat pattern for each service)
AUTH_SVC_DB_NAME=auth_db
AUTH_SVC_DB_USERNAME=auth_user
AUTH_SVC_DB_PASSWORD=localpass

PARENT_SVC_DB_NAME=parent_db
PARENT_SVC_DB_USERNAME=parent_user
PARENT_SVC_DB_PASSWORD=localpass

CENTER_SVC_DB_NAME=center_db
CENTER_SVC_DB_USERNAME=center_user
CENTER_SVC_DB_PASSWORD=localpass

ASSESS_SVC_DB_NAME=assess_db
ASSESS_SVC_DB_USERNAME=assess_user
ASSESS_SVC_DB_PASSWORD=localpass

PSYCH_SVC_DB_NAME=psych_db
PSYCH_SVC_DB_USERNAME=psych_user
PSYCH_SVC_DB_PASSWORD=localpass
```

HikariCP connection pool (one set per service — same pattern):

```
AUTH_SVC_DB_POOL_MAX_SIZE=10
AUTH_SVC_DB_POOL_MIN_IDLE=2
AUTH_SVC_DB_CONNECTION_TIMEOUT_MS=30000
AUTH_SVC_DB_IDLE_TIMEOUT_MS=600000
# Repeat for PARENT_SVC_, CENTER_SVC_, ASSESS_SVC_, PSYCH_SVC_
```

Redis

```
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=localpass     # required — must match what Docker starts Redis with
REDIS_SSL_ENABLED=false      # MUST be false for local plaintext Redis
```

Kafka (KRaft — no ZooKeeper)

```
KAFKA_BOOTSTRAP_SERVERS=localhost:9092
KAFKA_SCHEMA_REGISTRY_URL=http://localhost:8081
KAFKA_NODE_ID=1
KAFKA_CONTROLLER_QUORUM_VOTERS=1@kafka:9093
KAFKA_EXTERNAL_PORT=9092
KAFKA_AUTO_CREATE_TOPICS=true
KAFKA_DEFAULT_PARTITIONS=1
KAFKA_DEFAULT_REPLICATION_FACTOR=1
KAFKA_LOG_RETENTION_HOURS=168
```

Kafka topic names (required — no defaults in code):

```
KAFKA_TOPIC_AUTH_EVENTS=auth-events
KAFKA_TOPIC_AUDIT_IMMUTABLE=audit-immutable
KAFKA_TOPIC_NOTIFICATION_SEND=notification-send
KAFKA_TOPIC_PSYCH_EVENTS=psych-events
KAFKA_TOPIC_ASSESS_EVENTS=assess-events
KAFKA_TOPIC_CENTER_EVENTS=center-events
KAFKA_TOPIC_PARENT_EVENTS=parent-events
```

Kafka consumer group IDs (required — one per service):

```
AUTH_SVC_KAFKA_CONSUMER_GROUP=auth-svc-group
PARENT_SVC_KAFKA_CONSUMER_GROUP=parent-svc-group
CENTER_SVC_KAFKA_CONSUMER_GROUP=center-svc-group
ASSESS_SVC_KAFKA_CONSUMER_GROUP=assess-svc-group
PSYCH_SVC_KAFKA_CONSUMER_GROUP=psych-svc-group
AI_GATEWAY_SVC_KAFKA_CONSUMER_GROUP=ai-gateway-svc-group
```

JWT — see RSA Key Generation below for key paths:

```
JWT_PRIVATE_KEY_PATH=           # absolute path to generated PEM (see next section)
JWT_PUBLIC_KEY_PATH=            # absolute path to generated PEM (see next section)
JWT_ACCESS_TOKEN_EXPIRY_SECONDS=900
JWT_REFRESH_TOKEN_EXPIRY_SECONDS=604800
JWT_ISSUER=http://localhost:8081
JWT_JWKS_URI=http://localhost:8081/.well-known/jwks.json
```

Argon2id (OWASP 2024 recommended):

```
ARGON2_MEMORY_COST=65536
ARGON2_ITERATIONS=3
```

```
ARGON2_PARALLELISM=2
ARGON2_SALT_LENGTH=16
ARGON2_HASH_LENGTH=32
```

OTP (TOTP — RFC 6238):

```
OTP_EXPIRY_SECONDS=300
OTP_MAX_ATTEMPTS=3
OTP_LENGTH=6
```

Observability:

```
LOG_LEVEL_ROOT=INFO
LOG_LEVEL_APP=DEBUG
ACTUATOR_ENDPOINTS=health,info,metrics
APP_ENVIRONMENT=local
OTEL_SAMPLING_PROBABILITY=1.0
OTEL_EXPORTER_OTLP_ENDPOINT=     # leave blank — traces are dropped locally
```

Service application names (used as `spring.application.name`):

```
API_GATEWAY_NAME=api-gateway
AUTH_SVC_NAME=auth-svc
PARENT_SVC_NAME=parent-svc
CENTER_SVC_NAME=center-svc
ASSESS_SVC_NAME=assess-svc
PSYCH_SVC_NAME=psych-svc
AI_GATEWAY_SVC_NAME=ai-gateway-svc
```

Downstream URIs (used by api-gateway routing):

```
AUTH_SVC_URI=http://localhost:8081
PARENT_SVC_URI=http://localhost:8082
CENTER_SVC_URI=http://localhost:8083
ASSESS_SVC_URI=http://localhost:8084
PSYCH_SVC_URI=http://localhost:8085
AI_GATEWAY_SVC_URI=http://localhost:8086
AI_GATEWAY_BASE_URL=http://localhost:8086
```

Gateway rate limiter (start with permissive values locally):

```
GATEWAY_RATE_LIMIT_REPLENISH_RATE=100
GATEWAY_RATE_LIMIT_BURST_CAPACITY=200
GATEWAY_RATE_LIMIT_REQUESTED_TOKENS=1
```

Resilience4j (circuit breakers + rate limiter):

```
R4J_CB_AI_WINDOW_SIZE=10
R4J_CB_AI_FAILURE_THRESHOLD=50
R4J_CB_AI_WAIT_DURATION=5s
R4J_CB_AUTH_WINDOW_SIZE=10
R4J_CB_AUTH_FAILURE_THRESHOLD=50
R4J_CB_AUTH_WAIT_DURATION=5s
R4J_CB_CAPTCHA_WINDOW_SIZE=10
R4J_CB_CAPTCHA_FAILURE_THRESHOLD=50
R4J_CB_CAPTCHA_WAIT_DURATION=5s
R4J_TL_AI_TIMEOUT=30s
R4J_RL_LOGIN_LIMIT=10
R4J_RL_LOGIN_REFRESH_PERIOD=1s
R4J_RL_LOGIN_TIMEOUT=0
```

Dev GUI tool ports:

```
KAFKA_UI_PORT=9000
KAFKA_UI_CLUSTER_NAME=local
PGADMIN_EMAIL=admin@edutech.local
PGADMIN_PASSWORD=localpass
PGADMIN_PORT=9001
REDIS_COMMANDER_PORT=9002
```

Variables that can be deferred (leave blank unless you are testing the specific feature):

VARIABLE	FEATURE GATED
`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `OLLAMA_BASE_URL`	AI/LLM features in `ai-gateway-svc`
`HCAPTCHA_SITE_KEY`, `HCAPTCHA_SECRET_KEY`, `HCAPTCHA_VERIFY_URL`	hCaptcha in `auth-svc`
`TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_VERIFY_SERVICE_SID`	SMS OTP in `auth-svc`
`MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`	Email OTP in `auth-svc`
`FIREBASE_PROJECT_ID`, `FIREBASE_SERVICE_ACCOUNT_KEY_PATH`	Push notifications in `parent-svc`
`VAULT_URI`, `VAULT_TOKEN`, `VAULT_NAMESPACE`	HashiCorp Vault — production only

VARIABLE	FEATURE GATED
`PSYCH_AI_SVC_BASE_URL`	Python AI sidecar calls from `psych-svc`
CAT engine vars (`CAT_MIN_QUESTIONS`, etc.)	Computer Adaptive Testing in `assess-svc`
STOMP relay vars (`ASSESS_SVC_WS_*`)	WebSocket exam sessions in `assess-svc`

RSA Key Generation (JWT)

The platform uses **RS256 asymmetric signing**. `auth-svc` signs JWTs with an RSA-2048 private key; all other services verify tokens using the public key only. No shared secrets are distributed.

Generate a key pair with OpenSSL (pre-installed on macOS; available via `brew install openssl` or your system package manager):

```
# Store keys outside the repository — never commit private keys
mkdir -p ~/.edutech-keys

# Generate RSA-2048 private key
openssl genrsa -out ~/.edutech-keys/jwt-private.pem 2048

# Extract the public key
openssl rsa -in ~/.edutech-keys/jwt-private.pem -pubout -out ~/.edutech-keys/jwt-public.pem
```

Set the absolute paths in `.env`:

```
JWT_PRIVATE_KEY_PATH=/Users/your-username/.edutech-keys/jwt-private.pem
JWT_PUBLIC_KEY_PATH=/Users/your-username/.edutech-keys/jwt-public.pem
```

Use absolute paths. The JVM resolves these paths relative to the process working directory, and absolute paths eliminate ambiguity across run configurations.

Verify the generated files look correct:

```
head -1 ~/.edutech-keys/jwt-private.pem   # should print: -----BEGIN RSA PRIVATE KEY-----
head -1 ~/.edutech-keys/jwt-public.pem    # should print: -----BEGIN PUBLIC KEY-----
```

Security: Never place PEM files inside the repository directory. `.gitignore` excludes `*.pem` files, but the safest approach is to store them outside the project tree entirely. Regenerating keys invalidates all

previously issued tokens immediately.

Starting Infrastructure

All local infrastructure is defined in `infrastructure/docker/docker-compose.yml`. It reads values from `.env` via the `--env-file` flag.

```
cd infrastructure/docker

# Start all containers in detached mode
docker compose --env-file ../../.env up -d

# Verify containers are healthy (wait ~60 seconds after startup)
docker compose --env-file ../../.env ps

# Return to project root
cd ../..
```

What starts:

CONTAINER	IMAGE	DEFAULT PORT	PURPOSE
`edutech-postgres`	`timescale/timescaledb-ha:pg16-latest`	`5432`	PostgreSQL 16 with pgvector and TimescaleDB extensions built in
`edutech-redis`	`redis:7-alpine`	`6379`	Session store, JWT token blacklist, rate limiter backend for api-gateway
`edutech-kafka`	`bitnami/kafka:3.6`	`9092`	Event bus — KRaft mode, no ZooKeeper required
`edutech-kafka-ui`	`provectuslabs/kafka-ui:latest`	`$KAFKA_UI_PORT`	Browser Kafka management (dev only)
`edutech-pgadmin`	`dpage/pgadmin4:latest`	`$PGADMIN_PORT`	PostgreSQL GUI (dev only)
`edutech-redis-commander`	`rediscommander/redis-commander:latest`	`$REDIS_COMMANDER_PORT`	Redis key browser (dev only)

Kafka runs in **KRaft mode** — a single container acts as both controller and broker. `KAFKA_AUTO_CREATE_TOPICS=true` means topic creation is handled automatically on first publish; no manual `kafka-topics.sh` calls are needed for local development.

Teardown (keeps volumes — data is preserved on next `up -d`):

```
docker compose --env-file ../../.env down
```

Full reset (deletes all persisted data — use when you need a clean slate):

```
docker compose --env-file ../../.env down -v
```

Database Initialization

Flyway runs automatically on each service startup. There is no manual SQL to execute.

Each service:

1. Connects to its own named database (e.g., `auth_db`, `center_db`).
2. Creates its dedicated schema (e.g., `auth_schema`, `center_schema`) if it does not exist.
3. Runs all migration scripts from `services/{svc}/src/main/resources/db/migration/`.
4. Records applied migrations in `flyway_schema_history` within the schema.

The `timescale/timescaledb-ha:pg16-latest` image bundles both `pgvector` and `TimescaleDB` — no manual extension installation is required.

To reinitialize a service schema (e.g., after a destructive schema change):

```
# Drop the schema via psql or pgAdmin
DROP SCHEMA auth_schema CASCADE;
# Then restart auth-svc — Flyway recreates and repopulates it
```

Building the Project

The Maven multi-module build resolves modules in the order declared in the root `pom.xml`. Shared libraries are always built before services.

Build order (managed automatically by Maven):

1. `libs/common-security`
2. `libs/event-contracts`
3. `libs/test-fixtures`
4. `services/auth-svc`
5. `services/parent-svc`
6. `services/center-svc`
7. `services/assess-svc`
8. `services/psych-svc`

9. `services/ai-gateway-svc`
10. `services/api-gateway`

```
# Compile all modules (no tests)
mvn compile --no-transfer-progress

# Run all 75 tests (no infrastructure required — WireMock and in-process stubs used)
mvn test --no-transfer-progress

# Test a single service — --also-make builds its library dependencies first
mvn test -pl services/auth-svc --also-make --no-transfer-progress
mvn test -pl services/center-svc --also-make --no-transfer-progress
mvn test -pl services/parent-svc --also-make --no-transfer-progress
mvn test -pl services/assess-svc --also-make --no-transfer-progress
mvn test -pl services/psych-svc --also-make --no-transfer-progress
mvn test -pl services/ai-gateway-svc --also-make --no-transfer-progress
mvn test -pl services/api-gateway --also-make --no-transfer-progress

# Package a runnable JAR for a single service
mvn package -pl services/auth-svc --also-make -DskipTests --no-transfer-progress
# Output: services/auth-svc/target/auth-svc-1.0.0-SNAPSHOT.jar

# Override project version (used in CI)
mvn compile -Drevision=1.2.0
```

> **IntelliJ IDEA:** Import the root `pom.xml` as a Maven project. All modules are discovered automatically. Create Spring Boot run configurations per service and set environment variables by importing `.env` from the run configuration's **Environment variables** field.

Running Services

Each service reads its configuration entirely from environment variables — no hardcoded values exist in the codebase. Load `.env` into your shell before starting services.

Load environment variables

```
# bash / zsh
set -a && source .env && set +a
```

Option A: Maven Spring Boot plugin (development)

```
# Start auth-svc (after environment is loaded)
mvn -pl services/auth-svc spring-boot:run --no-transfer-progress
```

Open a separate terminal per service (or use IntelliJ's run dashboard to manage multiple).

Option B: Packaged JAR

```
# After packaging
java -jar services/auth-svc/target/auth-svc-1.0.0-SNAPSHOT.jar
```

Recommended startup order

Start services in this order to avoid transient connection errors on boot:

1. **Infrastructure** (Docker): confirm all containers are healthy.
2. `auth-svc` — other services depend on JWT validation via `common-security`.
3. `center-svc`, `parent-svc`, `assess-svc`, `psych-svc` — in any order.
4. `ai-gateway-svc` — requires `ANTHROPIC_API_KEY` if testing LLM features; can be skipped otherwise.
5. `api-gateway` — start last; it routes to all services above.

> **Virtual Threads note:** `spring.threads.virtual.enabled=true` is configured in every `application.yml`. On Java 17 this is a no-op (the property is recognised but Loom is preview-only). Full Project Loom throughput is available when running on Java 21+, which production containers use (Temurin 21).

Service Ports Reference

Default ports are defined in `.env.example` and resolve through `${SERVICE_PORT}` placeholders in each `application.yml`. Change them in `.env` if you have conflicts.

SERVICE	DEFAULT PORT	API-GATEWAY PATH PREFIX
`api-gateway`	8080	— (entry point)
`auth-svc`	8081	`/api/v1/auth/**`
`parent-svc`	8082	`/api/v1/parents/**`
`center-svc`	8083	`/api/v1/centers/**`
`assess-svc`	8084	`/api/v1/assessments/**`
`psych-svc`	8085	`/api/v1/psych/**`
`ai-gateway-svc`	8086	`/api/v1/ai/**`

SERVICE	DEFAULT PORT	API-GATEWAY PATH PREFIX
Python AI sidecar	8095	(internal — called by `ai-gateway-svc` only)

Verifying the Stack

Once infrastructure is healthy and services are running:

Health endpoints

```
curl -s http://localhost:8080/actuator/health | jq .   # api-gateway
curl -s http://localhost:8081/actuator/health | jq .   # auth-svc
curl -s http://localhost:8082/actuator/health | jq .   # parent-svc
curl -s http://localhost:8083/actuator/health | jq .   # center-svc
curl -s http://localhost:8084/actuator/health | jq .   # assess-svc
curl -s http://localhost:8085/actuator/health | jq .   # psych-svc
curl -s http://localhost:8086/actuator/health | jq .   # ai-gateway-svc
```

A fully healthy response:

```
{
  "status": "UP",
  "components": {
    "db":    { "status": "UP" },
    "redis": { "status": "UP" },
    "kafka": { "status": "UP" }
  }
}
```

Swagger UI

The `api-gateway` aggregates OpenAPI specs from all running downstream services into a single Swagger UI. Use the service selector dropdown to switch between service APIs.

```
http://localhost:8080/swagger-ui.html        # aggregated — all services
http://localhost:8081/swagger-ui.html        # auth-svc standalone
http://localhost:8082/swagger-ui.html        # parent-svc standalone
http://localhost:8083/swagger-ui.html        # center-svc standalone
http://localhost:8084/swagger-ui.html        # assess-svc standalone
http://localhost:8085/swagger-ui.html        # psych-svc standalone
```

Raw OpenAPI JSON for each service is available at `http://localhost:{port}/api-docs`.

Dev GUI Tools

TOOL	URL (DEFAULT PORTS)	PURPOSE
Kafka UI	`http://localhost:9000`	Browse topics, consumer group offsets, message payloads
pgAdmin	`http://localhost:9001`	PostgreSQL GUI — log in with `PGADMIN_EMAIL` / `PGADMIN_PASSWORD`
Redis Commander	`http://localhost:9002`	Browse Redis keys, inspect token blacklist and rate limiter state

Connecting pgAdmin to local PostgreSQL:

1. Open `http://localhost:9001` and log in.
2. Right-click **Servers** > **Register** > **Server**.
3. **General** tab: Name = `EduTech Local`
4. **Connection** tab: Host = `host.docker.internal` (macOS/Windows) or Docker bridge IP (Linux), Port = `5432`, Username = `POSTGRES_ROOT_USER`, Password = `POSTGRES_ROOT_PASSWORD`.

Common Issues and Fixes

Port conflict on startup

Symptom: `Address already in use: bind` in Docker or service logs.

Fix: Find the process occupying the port and stop it, or change the port assignment in `.env`.

```
lsof -i :5432    # find what is using port 5432
kill -9 <PID>
```

Redis connection refused

Symptom: `io.lettuce.core.RedisConnectionException: Unable to connect to localhost:6379`

Causes and fixes:

1. Redis container is not running — check `docker compose ps`.
2. `REDIS_SSL_ENABLED=true` is set locally — the local Redis container runs without TLS. Set `REDIS_SSL_ENABLED=false` in `.env`.
3. `REDIS_PASSWORD` mismatch — the password in `.env` must match the one used when the container started. If you changed it after the container was created, run `docker compose down -v && docker compose up -d` to recreate with the new password.

Kafka timeout at service startup

Symptom: `org.apache.kafka.common.errors.TimeoutException: Topic X not present in metadata` or producer/consumer errors immediately after service boot.

Cause: The Kafka KRaft broker needs 30–60 seconds to complete leader election after the container starts.

Fix: Wait for Kafka to report `healthy` before starting services:

```
docker compose --env-file ../../.env ps

# Wait until edutech-kafka shows: (healthy)
```

Manually confirm Kafka is ready:

```
docker exec edutech-kafka kafka-topics.sh --bootstrap-server localhost:9092 --list
```

JWT validation fails — signature mismatch

Symptom: `io.jsonwebtoken.security.SignatureException: JWT signature does not match locally computed signature`

Causes and fixes:

1. `JWT_PUBLIC_KEY_PATH` on the validating service points to a different key than `JWT_PRIVATE_KEY_PATH` on `auth-svc`. Both must reference the same RSA key pair generated in the RSA Key Generation step.
2. The PEM file path is wrong or the file is not readable — verify: `cat $JWT_PUBLIC_KEY_PATH`.
3. Keys were regenerated after tokens were issued — all existing tokens are immediately invalid. Obtain a fresh token via the login endpoint.

`Could not resolve placeholder` error at startup

Symptom: `java.lang.IllegalArgumentException: Could not resolve placeholder '${SOME_VAR}' in value "${SOME_VAR}"`

Cause: A required environment variable is missing or blank in `.env`.

Fix: Find the variable in `.env.example`, add it to `.env` with an appropriate local value, then restart the service. Use the tables in the Environment Setup section to identify required vs. optional variables.

Flyway migration error — non-empty schema without history table

Symptom: `FlywayException: Found non-empty schema(s) "auth_schema" without schema history table!`

Cause: Tables were manually created in the schema before Flyway ran for the first time.

Fix: Drop the schema and let Flyway recreate it cleanly on the next service startup:

```
# Via psql or pgAdmin:
DROP SCHEMA auth_schema CASCADE;
# Restart the service — Flyway recreates and migrates the schema.
```

Alternatively, do a full infrastructure reset:

```
cd infrastructure/docker
docker compose --env-file ../../.env down -v
docker compose --env-file ../../.env up -d
```

---

pgvector extension not found

Symptom: Flyway migration fails with `ERROR: extension "vector" does not exist`

Cause: A non-pgvector-enabled Postgres image is in use, or the container was replaced.

Fix: Confirm `infrastructure/docker/docker-compose.yml` uses `timescale/timescaledb-ha:pg16-latest`. If you must use a standard Postgres image, install pgvector manually:

```
docker exec -it edutech-postgres psql -U ${POSTGRES_ROOT_USER} -c "CREATE EXTENSION IF NOT EXISTS
vector;"
```

`JAVA_HOME` not set correctly — wrong JDK used

Symptom: Maven compiles with an unexpected JDK version, or `java -version` shows a different version than `mvn -version`.

Fix:

```
# macOS with Temurin 17
export JAVA_HOME=$(/usr/libexec/java_home -v 17)
export PATH="$JAVA_HOME/bin:$PATH"

java -version    # must show 17.x
mvn -version     # must show Java version: 17.x
```

Add these exports to `~/.zshrc` or `~/.bashrc` to persist across terminal sessions.